AUTONOMOUS MULTI-AGENT SYSTEMS • ENTERPRISE COMPLIANCE AI

RegRadar: Multi-Agent Regulatory Change Radar

Automated Regulatory Ingestion, Control Mapping, Adversarial Gap Validation & Remediation Planning

Domain: Fintech / Payment Systems (RBI • MeitY) **Architecture:** LangGraph Multi-Agent StateGraph **Stack:** Python 3.12, FastAPI, React 19, SQLite/Postgres

1. EXECUTIVE SUMMARY & PROBLEM STATEMENT

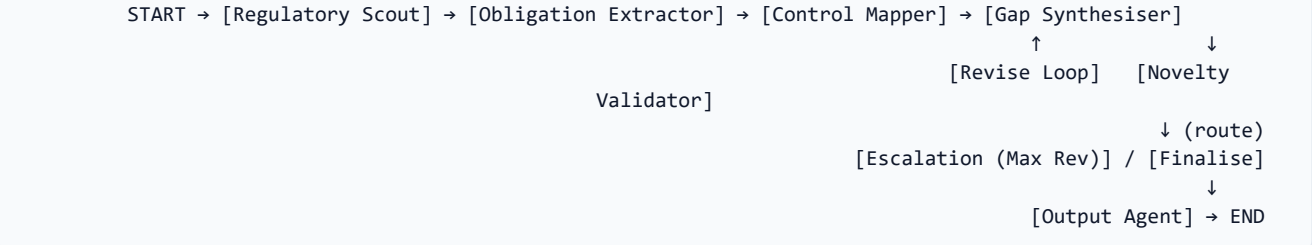
Modern digital financial institutions operate under strict, rapidly evolving regulatory regimes issued by authorities such as the Reserve Bank of India (RBI), the Securities and Exchange Board of India (SEBI), and the Ministry of Electronics and Information Technology (MeitY). Compliance teams face three acute challenges:

1. **Information Velocity & Latency:** New Master Directions and circulars require hundreds of engineering and policy hours to manually decompose into atomic obligations.
2. **Quantitative Shortfall Blindspots:** Generalist analysis often confirms qualitative alignment (e.g., "incident response policy exists") while missing precise quantitative shortfalls (e.g., regulation mandating 6-hour regulatory reporting while internal policy operates on a 24-hour SLA).
3. **LLM Hallucination & Over-Optimism:** Naive single-prompt LLM agents suffer from hallucinations, ungrounded citations, and sycophantic acceptance of superficial compliance, creating severe regulatory exposure.

RegRadar solves this through a cooperative, adversarial multi-agent architecture built on **LangGraph**. Six specialized autonomous agents retrieve circulars, extract atomic obligations, compute coverage matrices against internal controls, synthesize gaps, rigorously stress-test each gap across **nine authoritative validation checks** in an adaptive feedback loop, and prioritize remediation actions.

2. SYSTEM ARCHITECTURE & THE SIX SPECIALIZED AGENTS

Agents in RegRadar do not communicate point-to-point; all interactions occur through a strongly-typed shared state (`RegRadarState`) with chronological execution trace logging:



#	Agent Name	Primary Role & Methodology	Output Artifact
1	**Regulatory Scout**	Ingests circulars (synthetic demo corpus, pasted documents, or live RBI/SEBI RSS feeds) and scores relevance against organization profile keywords.	`documents`

#	Agent Name	Primary Role & Methodology	Output Artifact
2	**Obligation Extractor**	Decomposes clauses into atomic obligations; identifies deontic modals (*shall/must* → mandatory, *should* → recommended, *may* → permissive) and regex-extracts quantitative parameters (deadlines, retention, audit frequencies). Enforces strict verbatim anti-hallucination.	`obligations`
3	**Control Mapper**	Evaluates coverage via a hybrid metric (0.65 × TF-IDF Cosine + 0.35 × Keyword Overlap). Flags quantitative parameter shortfalls against internal controls.	`coverage` matrix
4	**Gap Synthesiser**	Drafts concrete compliance gaps in Round 0. In subsequent rounds, consumes structured validator feedback to revise, withdraw, or maintain proposals.	`gaps` proposals
5	**Novelty Validator**	Adversarial quality gate. Re-derives facts independently across 9 deterministic rules (evidence grounding, novelty, duplicate check, severity calibration). Emits structured feedback codes.	`validation_decisions` & `pending_feedback`
6	**Output Agent**	Applies a quantitative multi-factor risk formula ($R = S \times U \times E \times C$), determines P1/P2/P3 SLAs and departmental owners, and writes executive and technical Markdown reports.	`remediation_plan` & `report`

3. KEY ENGINEERING HIGHLIGHTS & INNOVATIONS

- **Adaptive Feedback Loop with Machine-Readable Contracts:** When the Novelty Validator rejects a proposal, it outputs structured issues containing `code`, `field`, `expected`, and `message`. The Gap Synthesiser processes this machine-readable contract programmatically to correct errors, withdraw redundant gaps, or mount defensible counter-arguments.
- **Independent Validation Authority:** Deterministic business rules always hold absolute authority over language models. LLMs are leveraged for semantic extraction and drafting, but cannot bypass or override the validator's grounding checks.
- **Escalation Handling:** The feedback cycle is formally bounded by `max_revisions`. If an adversarial impasse occurs (e.g., an agent repeatedly defends a non-mandatory recommendation), the graph breaks the cycle and escalates the specific gap to human reviewers with a complete rationale.
- **Risk-Calibrated Remediation Formula:** Remediation items are dynamically scored using: $$\text{Risk Score} = \text{Severity Weight (2-10)} \times \text{Urgency Factor (1.0-1.6)} \times \text{Exposure Factor (1.0-1.25)} \times \text{Classification Factor (0.5-1.0)}$$

4. TECHNOLOGY STACK & VERIFICATION

- **Backend:** Python 3.12, FastAPI, LangGraph 1.2+, Pydantic v2.14, Uvicorn, SQLite/PostgreSQL (Neon).
- **Frontend:** React 19, TypeScript, Vite, CSS Design System, Responsive Multi-Page SPA.
- **Quality Assurance:** 64 automated pytest tests covering unit components, routing decisions, feedback convergence, and API contracts. End-to-end verified via `scripts/verify.ps1`.